□ TypeScript SSOT & Strict Typing Guide

Este documento define el estándar de ingeniería para el código TypeScript en el proyecto, emulando la seguridad de un lenguaje de tipado estático como Go o Rust.

---

1. Manifiesto SSOT (Single Source of Truth)

El proyecto adopta SSOT v2.0, lo que significa que el compilador de TypeScript es el guardián absoluto de la lógica de negocio. Se prohíbe cualquier patrón que inyecte incertidumbre en el runtime.

□ Reglas Prohibidas (NUNCA)

- 0.1 PROHIBIDO any: Su uso es motivo de rechazo inmediato. Usar unknown con type guards.
- 0.2 PROHIBIDO undefined implícito: Usar null explícito o la monada Option<T>.
- 0.3 PROHIBIDO NaN sin validación: Todo parseo numérico debe validarse con Number.isNaN().
- 0.4 PROHIBIDO throw para lógica de negocio: Las funciones deben retornar Result<T, E>. El throw se reserva para fallos catastróficos (panics).
- 0.5 PROHIBIDO inicialización parcial: No usar {} as Type. Las interfaces deben inicializarse de forma atómica.

□ Reglas Obligatorias (SIEMPRE)

- 1.1 Patrón Result<T, E>: Para todas las operaciones que pueden fallar.
- 1.2 Patrón Option<T>: Para valores opcionales, forzando el desempaquetado seguro.
- 1.3 Branded Types: Todos los IDs de dominio (ProviderID, PatientID) deben estar brandeados para evitar mezclas accidentales.
- 1.4 "Parse, don't validate": Uso estricto de Zod con .strict() en todas las fronteras de I/O (Webhooks, API, DB).
- 1.5 readonly por defecto: Favorecer la inmutabilidad de los datos.

---

2. Configuración de Enforcement

El cumplimiento de estas reglas se automatiza mediante:

- TSConfig Strictest: Uso de noUncheckedIndexedAccess y exactOptionalPropertyTypes.
- ESLint Strict: Reglas @typescript-eslint/no-explicit-any y explicit-function-return-type configuradas como ERROR.
- Zod Guardians: Esquemas en internal/schemas/ que actúan como aduana de datos.

---

3. Estado de la Migración (Go □ TS)

La migración a SSOT v2.0 está 100% completada para los módulos críticos:

Módulo	Estado	Patrones Aplicados
Domain Types	□ 100%	Branded IDs, Result/Option Monad...
AI Agent	□ 100%	Zod Boundary Parsing, Semantic Lo...
Transactional Core	□ 100%	Serializable TX handling, Error map...
Notifications	□ 100%	Retry loop con backoff exponencial.

Ejemplos Maatros

Error Handling (Result Pattern)

```typescript
function parseBooking(raw: unknown): Result<Booking, Error> {
const parsed = BookingSchema.safeParse(raw);
if (!parsed.success) return err(new Error("Invalid format"));
return ok(parsed.data);
}
```

Branded IDs

```typescript
type ProviderID = Brand<string, "ProviderID">;
const pid = "uuid" as ProviderID; // Compila
const sid: ServiceID = pid;       // ERROR DE COMPILACIÓN
```

Referencias Detalladas

- docs/migration/MIGRATION_FINAL_STATUS.md
- docs/migration/MIGRATION_VERIFICATION_REPORT.md